Modeling Human Activity States Using Hidden Markov Models

link to video of data collection: https://canva.link/bddekxengm16x8d

1. Background and motivation

Wearable health monitors and ordinary smartphones carry accelerometers and gyroscopes that stream continuous motion data all day long, and with that knowing what a person is actually doing with that movement, whether they are walking, standing, jumping or sitting completely still, is useful in a lot of places. even fitness applications use it to count activity minutes, elderly care systems use it to detect falls, and physiotherapists use it to check whether a patient is doing the exercises they were given to do.

The difficult thing is that the activity it self is never measured directly, so like what the sensors gives us is a noisy stream of numbers, and the activity behind those numbers has to be inferred. There is also a time structure to the problem that a plain classifier would throw away, like If somebody was walking one second ago, they are very likely still walking now, because people do not switch between activities many times per second. A hidden markov model captures both of these things at once. The activity is the hidden state, the sensor readings are the observations, and the model learns how states follow one another over time as well as what each state tends to look like in the data.

2. Data collection and preprocessing

2.1 Recording setup

Data was collected with the Sensor Logger app running on two phones at the same time, one phone was iPhone 8 Plus and a second phone was Samsung A17, both phones were set to a sampling rate of 50 Hz, and the accelerometer, gravity and gyroscope sensors were switched on while everything else was left off. by running both phones together for every take gave me two advantages, first It doubled the amount of data collected per round, and it gave me a direct way to check that the two devices were measuring the same thing, since any disagreement between them would show up immediately when the two files were compared.

2.2 Training and test sessions

The training set was recorded in my living room, it was ten rounds recorded for each of the four activities, and because two phones were running each round, the output was 20 samples per activity and 80 training files in total. every sample ran between 10.4 and 12.6 seconds, which is a bit longer than the 5 to 10 seconds suggested in the brief and gives a little more signal per file.

The test set was recorded in a separate session in the bedroom. a different room, a different time, and the phones were held in the other hand. Three rounds per activity were recorded, giving 6 samples per activity and 24 test files, and none of these recordings were used at any point during training, and the clustering and normalisation values

were taken from the training data only. short videos was also filmed for each activity showing the full body and the phone being carried during the recording.

2.3 Harmonising the two phones

Sensor Logger writes each sensor into its own file with its own timestamps, so the three streams have to be brought together before anything else can happen. A conversion script merges the accelerometer, gravity and gyroscope files by matching each reading to the nearest timestamp in the others, then resamples the merged result onto a fixed 50 Hz grid. This resampling step is what actually harmonises the two phones, because although both were set to 50 Hz, their raw timestamps do not line up with each other and the real intervals drift slightly.
As a check on consistency, the gravity magnitude was measured across every file from both devices. It came out at 9.807 on all 104 recordings, which confirmed that the two phones were calibrated the same way and that the merge and resample had not distorted anything.

Activity	Train files (living room)	Test files (bedroom)	Acceleration mean	Gyroscope mean
Standing	20	6	0.13 to 0.24	0.05 to 0.08
Walking	20	6	1.43 to 2.39	0.94 to 1.36
Jumping	20	6	10.8 to 16.5	2.02 to 2.68
Still	20	6	0.02 to 0.04	0.001 to 0.002

Table 1. Dataset summary.

3. Feature extraction

Fifty readings per second is way more detail than the model needs, so i cut each recording into windows of 64 samples, about 1.3 seconds, with 32 samples of overlap. the overlap stops a short event from falling between two windows and getting lost.

Every window got reduced to 14 numbers. the time domain ones are the mean and standard deviation of the acceleration magnitude, the signal magnitude area, the correlations between the x, y and z axes, and the mean and standard deviation of the gyroscope magnitude. the frequency domain ones come from the of the acceleration magnitude, and they are the dominant frequency, the spectral energy and the spectral entropy. dominant frequency is useful for walking since walking has a steady rhythm, and spectral entropy is low when movement is repetitive and high when it is messy. the last three are the gravity vector components, which say how the phone is tilted. i added tilt later because standing and still are both almost motionless and orientation is really the only thing separating them. this gave me 1256 training windows and 387 test windows.

The features sit on very different scales, dominant frequency is around two while spectral energy runs into the thousands, so i standardised each one with ard deviation came from the training data only and i applied those same values to the test data so nothing leaked in. and since the model needs discrete
observations, k means with 12 clusters was fs and each window got labelled with its cluster number.

4. HMM setup and implementation

I wrote the model from scratch in numpy, no machine learning library. the hidden states are the four activities and the observations are the 12 cluster numbers.the chance of moving from one activity to another between windows, the emission matrix the chance of seeing each symbol in a given activity, and the initial probabilities the chance of starting out in There are three algorithms normally tied to ly used two. the forward algorithm gives the probability of a whole sequence but does not tell you what was happening at any particular moment, so it was not enough. baum welch learns the matrices from gle most likely path, so you get one predicted activity per window, and since sensitivity, specificity and accuracy are all measured per window that one i needed.

Training happens in two stages. i counted the labelled training data with add one smoothing so nothing is ever exactly zero, then refined them with baum welch. counting first matters because baum welch is unsupervised, run on its own it will arrange no way to know which one is walking. and since each recording only holds one activity from start to finish, i stitched random recordings into ten longer sessions so the model could see real transliterations at a log likelihood of -774.03. two small things were needed to make it work on sequences this long, the forward and backward passes are scaled at every step and viterbi runs in log space, otherwise to zero and the whole thing collapses.

5. Results and interpretation

The model was evaluated on the 387 windows from the bedroom test session. It reached an overall accuracy of 95.1 percent across all windows.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Standing	96	0.927	0.979	0.966
Walking	95	1.000	0.973	0.979
Jumping	99	0.939	1.000	0.984
Still	97	0.938	0.983	0.972

Table 2. Evaluation on the unseen bedroom session.

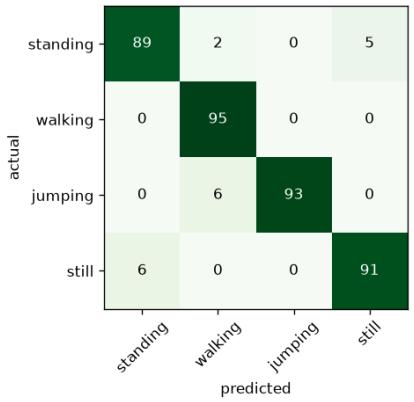


Figure 1. Confusion matrix on the unseen bedroom data.

Walking came out perfect, sensitivity of 1.000 and not a single window missed. that fits what the signal looks like, walking has a steady rhythm that gives it a dominant frequency nothing else in the set comes close to. jumping was never confused with either of the motionless activities and got a specificity of 1.000, so nothing else was ever mistaken for jumping either. its only errors were six windows read as walking, and those are almost certainly the moments at the start and end of a recording before i had properly settled into a rhythm.

What is left is standing against still, five windows one way and six the other. that is the hardest pair in the whole dataset and the result does not surprise me, both are basically motionless and the only real difference is that standing has a bit of body sway with the phone held upright, while still is just the phone sitting untouched on a surface.

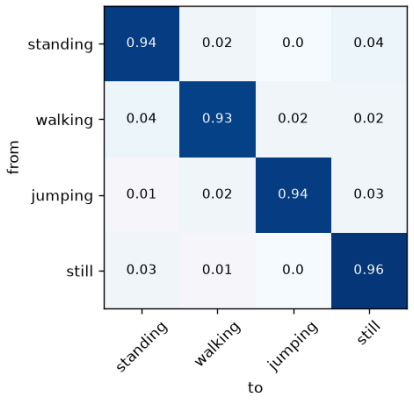


Figure 2. Learned transition probabilities between the four activity states.

The learned transition matrix has 0.939 for standing, 0.925 for walking, 0.935 for jumping and 0.958 for still along the diagonal. a diagonal that high is exactly what real behaviour should look like, an activity lasts way longer than the 1.3 seconds a single window covers, so the most likely thing to happen next is that you just keep doing it. if those values had come out low it would mean the model was flipping between states several times a second, which no human does, and that would point at noisy features rather than anything real. the biggest off diagonal values are

the ones connecting standing and still, which matches the confusion matrix and tells me the model finds those two the most interchangeable.

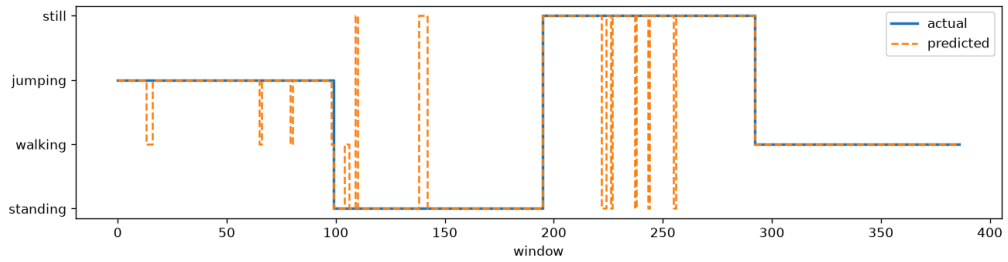


Figure 3. Viterbi decoded sequence on the unseen data, predicted activity against the true activity.

6. Discussion and conclusion

6.1 A problem worth recording

The first evaluation on the bedroom test set gave only 66.9 percent accuracy, with standing and still barely better than a coin flip, even though a holdout check within the living room data had given 96.8 percent. That gap was too large to be ordinary generalisation loss and pointed at something systematic. The cause turned out to be that iOS and Android report the gravity vector with opposite signs. Because the two phones were used interchangeably, the tilt feature was flipping direction from one file to the next and was effectively feeding the model random noise in exactly the three features that standing and still depended on.

Using the size of each gravity component rather than its signed value removed the inconsistency and raised accuracy from 66.9 percent to 95.1 percent. This is worth recording because it is a problem that only appears when more than one kind of device is used, and it would have been invisible if all the data had come from a single phone. Matching the sampling rate across devices, which the brief asks for, is necessary but it is not sufficient. The sign and unit conventions need checking too.

6.2 Sensor noise and sampling rate

Sensor noise was otherwise low and no recordings had to be discarded. The sampling rate of 50 Hz proved comfortable for all four activities, since even jumping, by far the most energetic of the four, peaks at only a few hertz, which sits well inside what 50 Hz can capture without aliasing.

6.3 Conclusion

A hidden markov model implemented from scratch in numpy, trained on 1256 windows of living room data and tested on a separate bedroom session, recognised four everyday activities with 95.1 percent overall accuracy and sensitivity above 0.92 for every activity. The learned transition matrix reflects realistic behaviour with strong self transitions, and the errors that remain are concentrated in the one pair of activities that is genuinely difficult to tell apart from motion alone. The result shows that reliable activity recognition does not need specialised hardware, just a phone, a sensible set of features and a model that respects the fact that activities persist over time.